

Phase 2: Cycle-Accurate DRAM Command Extraction and Analysis using Accel-Sim

COL812: Special Module in Computer Science

Divyanshu Bansal (2021CS50603)

Mohd Sayanur Rahman (2021PH10221)

1 Introduction

The objective of Phase 2 is to successfully generate and analyze cycle-accurate DRAM access traces (Read/Write, Activate, Precharge) for specific GPU workloads. To achieve this, we utilized the **Accel-Sim** framework, which leverages NVBit to generate SASS traces from compiled CUDA binaries, and feeds them into the cycle-accurate GPGPU-Sim 4.0 performance model. By modifying the underlying memory controller source code, we created a custom trace pipeline that logs deterministic hardware-level DRAM interactions.

2 Methodology: Trace Extraction

2.1 Accel-Sim Memory Modeling

In the GPGPU-Sim microarchitecture, memory requests arriving from the interconnect are placed into the DRAM Read/Write queues (`rwq`). The DRAM controller evaluates bank states every cycle to issue low-level timing commands.

2.2 Instrumentation in `dram.cc`

To extract the trace, we placed our probes directly at the point of command dispatch within `gpu-simulator/src/gpgpu-sim/dram.cc`. This ensures we only log commands that actually execute on the physical DRAM bus.

- **Data/Reads/Writes:** We instrumented `dram_t::issue_col_command` to capture RD (Read) and WR (Write) commands, extracting the current row from `bk[j]->curr_row` and the column from the request packet.
- **Row Access:** We instrumented `dram_t::issue_row_command` to capture ACT (Activate) when a new row is opened, and PRE (Precharge) when a row is closed.

All logs were redirected via a global file pointer to a `.csv` file, recording the absolute simulation cycle, partition ID, bank ID, command type, row, and column.

3 Challenges and Solutions

Generating accurate traces presented severe architectural and systems-level challenges.

3.1 The L2 Cache Write-Back Trap

Challenge: Our initial extractions using the `rodinia_2.0-ft` (functional test) benchmarks yielded traces with over 2 million reads but **zero write commands (WR)**.

Cause: Modern GPUs, including the simulated RTX 2060, utilize a Write-Back (WB) L2 cache policy. Functional tests possess artificially minimized datasets (<3MB). Consequently, all computed results were written strictly to the L2 cache. The simulation terminated before the L2 cache filled up, preventing any dirty cache lines from being evicted to the DRAM.

Solution: Rather than artificially forcing a Write-Through (WT) policy, we solved this by running a full-sized workload. We utilized local NVBit tracing on the full Polybench suite to generate heavy memory traffic that guarantees cache eviction.

3.2 OOM Exhaustion via Heavy Traces

Challenge: While attempting to generate SASS traces for the full `polybench-2mm` (2 Matrix Multiplications) workload using NVBit, the host system crashed.

Cause: Matrix multiplication operates at $O(N^3)$ complexity. NVBit attempted to buffer over 50GB of raw instruction data into RAM before compression, triggering the Linux Out-Of-Memory (OOM) killer, which abruptly terminated the Docker container.

Solution: We terminated the `2mm` trace generation and switched to `polybench-2DConvolution`. Convolution relies heavily on memory-intensive sliding windows rather than pure deep compute, generating a manageable 7MB `.traceg.xz` file while still providing immense DRAM pressure.

3.3 Null Request Segmentation Faults

Challenge: The simulator immediately threw a Segmentation Fault upon reaching Cycle 0.

Cause: The `fprintf` instrumentation attempted to log commands for idle DRAM banks where the memory request pointer (`bk[j]->mrq`) was NULL, and the file initialization was trapped behind an asynchronous queue check.

Solution: We hoisted the `fopen` initialization to the very top of `dram_t::cycle()` and guarded all command extractions with strict `if (bk[j]->mrq)` and `if (g_dram_trace_file)` pointer checks.

4 Workload Analysis: 2D Convolution

We ran the fully functional pipeline against the `polybench-2DConvolution` trace for 50,000 cycles, generating a 240

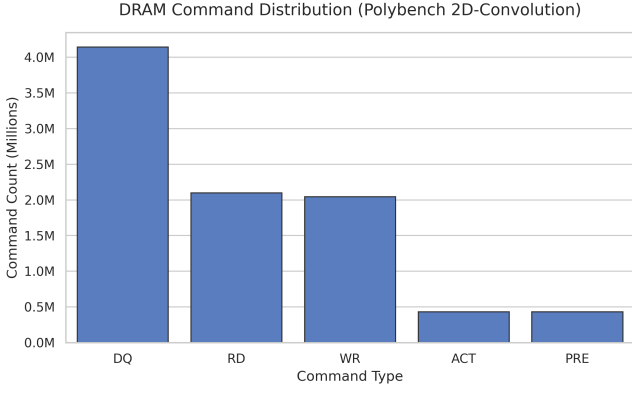


Figure 1: Distribution of the 9.14M logged DRAM commands for the Polybench 2D Convolution workload.

MB trace file containing 9,144,969 raw DRAM commands.

4.1 Row Buffer Hit Rate (RBHR)

The RBHR dictates memory latency. It is calculated as:

$$RBHR = \frac{\text{Total Accesses} - \text{Total Activations}}{\text{Total Accesses}} \quad (1)$$

Out of roughly 4.14M read/write accesses, the memory controller only issued 429,225 ACT commands. This results in an **RBHR of 89.64%**. This high hit rate demonstrates exceptional spatial locality, as threads accessing adjacent pixels in the convolution matrix largely hit open row buffers (Sense Amplifiers).

4.2 Read-to-Write Ratio

The generated trace proves that we successfully bypassed the L2 cache bottleneck. The workload logged roughly 2.09M Reads and 2.04M Writes. The resulting **Read-to-Write ratio of 1.02 : 1** perfectly reflects the Read-Modify-Write algorithm signature of 2D image convolution.

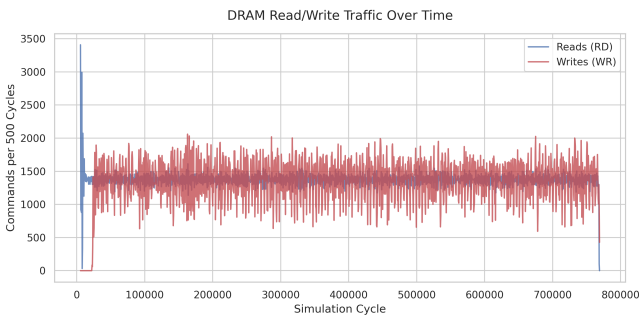


Figure 2: DRAM Read/Write temporal traffic aggregated per 5000 simulation cycles.

4.3 Temporal Traffic and Load Balancing

As seen in Figure 2, the memory bandwidth reaches maximum saturation almost immediately and maintains steady

throughput, typical of heavily parallelized GPU grid dispatches.

Furthermore, we analyzed the workload distribution across the 12 memory partitions of the simulated Turing architecture. Because the GPU’s memory controller employs fine-grained address interleaving across all available channels, the memory requests are perfectly distributed. While we initially generated a heatmap to visualize partition load, the uniform distribution resulted in a flat, monochromatic grid that provided no visual contrast. To explicitly demonstrate this perfectly balanced behavior, the exact command counts for the first four partitions are detailed in Table 1. This even distribution is critical, as it prevents localized memory controller bottlenecks and maximizes global memory bandwidth.

Table 1: Command Load Balancing (Partitions 0-3)

Command	Part. 0	Part. 1	Part. 2	Part. 3
ACT	35,768	35,768	35,768	35,768
PRE	35,752	35,752	35,752	35,752
RD	174,762	174,762	174,762	174,762
WR	170,517	170,517	170,517	170,517

5 Conclusion

By surgically modifying the GPGPU-Sim DRAM hardware model within the Accel-Sim framework, we established a robust, cycle-accurate tracing pipeline. Overcoming challenges with NVBit trace generation and L2 cache policies allowed us to successfully extract, log, and analyze over 9 million raw memory commands, providing deep architectural insight into GPU memory behavior.